



Faculté des Sciences Ben M'Sick, Université Hassan II-Casablanca
Master Data Sciences & Big Data
&
Master d'Excellence en Intelligence Artificielle



Année universitaire : 2025-2026

Pr. Said NOUH

Chapitre 1:

Caractéristiques du langage Python et sa puissance en sciences de données

- Python est **portable**. il fonctionne sur toutes les plateformes (Windows, Linux, macOS) sans modification majeure du code.
- Python convient aussi bien à des **scripts** d'une dizaine de lignes qu'à des **projets complexes** de plusieurs dizaines de milliers de lignes.
- La **syntaxe** de Python est très simple et, combinée à des **types de données évolués** (listes, dictionnaires,...), conduit à des programmes à la fois très compacts et très lisibles.
- Python gère ses ressources (mémoire, descripteurs de fichiers...) sans intervention du programmeur.
- Il n'y a **pas de pointeurs** explicites en Python.
- Python est **orienté-objet**. Il supporte l'**héritage multiple** et la **surcharge des opérateurs**.

Chapitre 1:

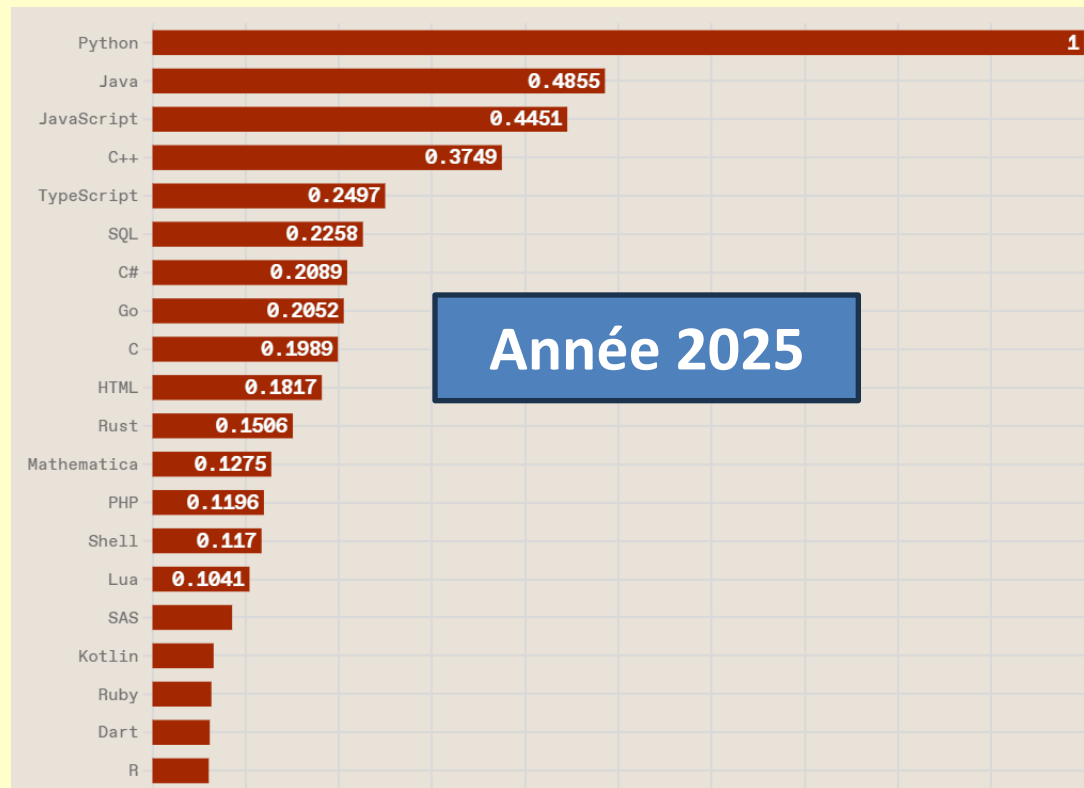
Caractéristiques du langage Python et sa puissance en sciences de données

- **Simplicité et lisibilité** : sa syntaxe claire et proche du langage naturel facilite l'apprentissage et la maintenance du code.
- **Polyvalence** : Python est utilisé pour le web, l'IA, les sciences de données, la cybersécurité, etc.
- **Grande communauté et écosystème riche** : des milliers de bibliothèques couvrent presque tous les besoins techniques.
- **Support communautaire massif**, ce qui accélère l'apprentissage et la résolution de problèmes.
- Python intègre un système d'**exceptions**, qui permettent de simplifier considérablement la gestion des erreurs.

Chapitre 1:

Caractéristiques du langage Python et sa puissance en sciences de données

- Python est un langage qui **continue** à **évoluer**, soutenu par une communauté d'utilisateurs actifs.
- L'Institute of Electrical and Electronics Engineers (IEEE) a publié son classement annuel des meilleurs langages de programmation. c'est Python qui a pris la tête du classement général en dépassant Java et C depuis plusieurs années.



Chapitre 1:

Caractéristiques du langage Python et sa puissance en sciences de données

Bibliothèques Riches et Performantes:

Python dispose d'une multitude de bibliothèques spécialement conçues pour la science des données :

- **NumPy** : Pour le calcul scientifique, notamment les opérations sur des tableaux multidimensionnels.
- **Pandas** : Pour la manipulation et l'analyse des données, avec des structures de données comme les DataFrames.
- **Matplotlib et Seaborn** : Pour la visualisation des données, permettant de créer des graphiques et des visualisations attrayantes.
- **Scikit-learn** : Pour l'apprentissage automatique, offrant des outils pour le prétraitement des données, l'évaluation des modèles et l'implémentation d'algorithmes de machine learning.
- **TensorFlow et PyTorch** : Pour l'apprentissage profond, ces bibliothèques permettent de construire et d'entraîner des modèles de réseaux neuronaux.
-

Chapitre 1:

Caractéristiques du langage Python et sa puissance en sciences de données

- **Interopérabilité:** Python peut facilement interagir avec d'autres langages et outils, ce qui le rend flexible pour intégrer différentes technologies dans un flux de travail de science des données. Par exemple, il peut être utilisé avec R, SQL, et des outils de big data comme Spark.
- **Support pour le Big Data:** Avec des bibliothèques comme **PySpark**, Python peut gérer de grandes quantités de données qui ne tiennent pas en mémoire, ce qui est essentiel pour les projets de big data.
- **Environnement de Développement Intégré: Jupyter Notebooks :** Cet environnement interactif permet aux scientifiques des données de créer et de partager des documents contenant du code exécutable, des visualisations et du texte. C'est particulièrement utile pour le prototypage et la présentation des résultats.
- **Visualisation de Données:** Les capacités de visualisation de Python, grâce à des bibliothèques comme **Matplotlib**, **Seaborn** et **Plotly**, permettent de créer des visualisations interactives et informatives qui aident à comprendre les données et à présenter les résultats de manière efficace.

Chapitre 2:

Rappel des bases du langage python

- Type de variables
 - entier (python: int) pour manipuler des entiers
 - réel (python: float) pour manipuler des nombres réels
 - booléen (python: bool) pour manipuler des valeurs booléennes
 - Caractère (python: str) pour manipuler des caractères alphabétiques ou numériques. On peut manipuler des chaînes de caractères permettant de représenter des mots ou des phrases.

Chapitre 2: Rappel des bases du langage python

Les opérateurs

- Les opérateurs: +, -, *, / Avec en plus pour les entiers div et mod, qui permettent respectivement de calculer une division entière et le reste de cette division.

par exemple : $11 \text{ div } 2$ vaut 5, $11 \text{ mod } 2$ vaut 1

En Python: $11//2$ vaut 5 et $11\%3$ vaut 2

- L'opérateur d'égalité :
pour les types simples il permet de savoir si les deux opérandes sont égales ou non. Il est représenté par le caractère ==
On a aussi l'opérateur d'inégalité : $\neq$ (en python !=)
- Et pour les types possédant un ordre les opérateurs de comparaison
 $<, \leq, >, \geq$

Chapitre 2: Rappel des bases du langage python

Les opérateurs

Opérateur	Fonction	Exemple
+	Addition	X+Y
-	Soustraction	X-Y
*	Multiplication	X*Y
/	Division	X/Y
^	Puissance	2^3
Mod	Reste division	17 mod 5 (=2)
xor	Ou exclusif bit à bit	5 xor 6 → 3

Langage Python
+
-
*
/ : division ; //division entière 6/4→1.5 ; 6//4→1
** : exemple 2**3→8
% : 17%5→2
^ : 5^6 → 3

- des opérateurs relationnels (de comparaison):

Opérateur	signification
=	Egal
<> ou ≠	Différent
<	Strictement inférieur
>	Strictement supérieur
<=	Inférieur ou égal
>=	Supérieur ou égal

Langage Python
==
!=
<
>
<=
>=

- des opérateurs logiques:

Opérateur	signification
NON	Non logique
OU	Ou inclusif
ET	Et logique

Langage Python
not
or
and

Chapitre 2: Rappel des bases du langage python

Les entrées / sorties

- ✓ **input()** et **print()** sont deux fonctions essentielles pour interagir avec l'utilisateur.
- ✓ La fonction **input()** permet de lire une donnée entrée au clavier : le programme s'arrête et attend que l'utilisateur tape quelque chose, puis renvoie ce texte sous forme de chaîne de caractères.
- ✓ On peut l'utiliser avec un message, par exemple:
`nom = input("Entrez votre nom : ")`.
- ✓ La fonction **print()** sert à afficher du texte ou des valeurs à l'écran ; elle peut montrer le résultat d'un calcul, le contenu d'une variable, ou un message destiné à l'utilisateur, comme `print("Bonjour", nom)`.
- ✓ Ensemble, **input()** et **print()** permettent de créer des programmes interactifs où l'utilisateur fournit des informations et reçoit un retour immédiat du programme.

Chapitre 2: Rappel des bases du langage python

ATTENTION à l'indentation en Python !!!!

Instruction conditionnelle :

```
if (conditions 1):
```

```
↔ .....
```

```
elif (conditions 2):
```

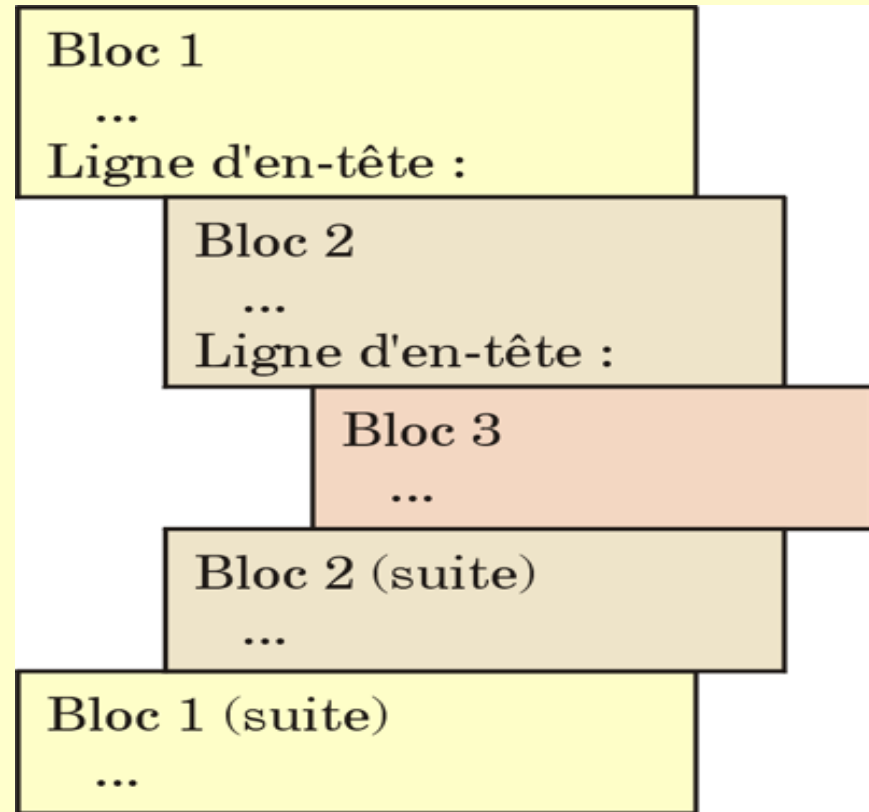
```
↔ .....
```

```
elif (conditions 3):
```

```
↔ .....
```

```
else :
```

```
↔ .....
```



Les blocs d'instructions sont toujours associés à une ligne d'en-tête contenant une instruction bien spécifique (if, elif, else, while, def, ...) ***se terminant par un double point.***

Chapitre 2: Rappel des bases du langage python

La boucle for

Il est fréquent que le nombre de répétitions soit connu à l'avance, et que l'on ait besoin d'utiliser le numéro de l'itération afin d'effectuer des calculs ou des tests. Le mécanisme permettant cela est la boucle Pour.

En Python:

```
for <cible> in <objet>:  
    <instructions>
```

Chapitre 2:

Rappel des bases du langage python

- La boucle **for** fonctionne sur les chaînes, les listes, les tuples et d'autres objets.
- La partie else: ... est optionnelle.
- L'instruction **range (a,b,pas)** permet de créer une liste **L** **croissante ou décroissante** d'entiers successifs . $a \in L$, $a + \text{pas} \in L$, $a + 2 * \text{pas} \in L$, $a + 3 * \text{pas} \in L$, Et $\forall x \in L: x < b$
- Exemple:

```
for i in range(10,20,3):  
    print (i);  
print("Sortie de la boucle avec i=",i);
```

Donne à l'exécution:

```
10  
13  
16  
19  
Sortie de la boucle avec i= 19
```

Exercice:

Donner le résultat du programme:

```
for i in range(10):  
    print(i)  
    i=i-1
```

Chapitre 2: Rappel des bases du langage python

La boucle for

```
fruits = ["pomme", "banane", "orange"]  
for fruit in fruits:  
    print(fruit)
```

```
liste = [(1, 2), (3, 4), (5, 6)]  
for a, b in liste:  
    print("Premier élément :", a, ", Deuxième élément :", b)
```

```
liste = [(1, 2), (3, 4), (5, 6)]  
for tup in liste:  
    print(tup[0], tup[1])
```

Chapitre 2:

Rappel des bases du langage python

La boucle while

La boucle while exécute un bloc tant que la condition de continuité est vérifiée. La particularité de ce while est qu'il peut y avoir un else optionnel dont le bloc sera exécuté lorsqu'on sort de la boucle.

```
while <condition> :  
    <instructions de la boucle>  
<instructions hors la boucle>
```

Exercice: quel est le résultat ?

```
x = 0.0; y=10  
while x!=1.0:  
    x+=0.1 ; y=y-1  
print("x à la sortie de la boucle x=",x, " et y=",y)
```

Exemple: programme qui compte le nombre de chiffres d'un nombre

```
a=(int)(input("Entrer un entier:"));  
if a==0:  
    print("le nombre de chiffres est:",1);  
else:  
    nbchif=0;  
    while a>0:  
        a=a//10;  
        nbchif=nbchif+1;  
    print("le nombre de chiffres est:",nbchif);
```

Chapitre 2: Rappel des bases du langage python

Les fonctions

Il arrivera souvent qu'une même séquence d'instructions doive être utilisée à plusieurs reprises dans un programme, et on souhaitera bien évidemment ne pas avoir à la reproduire systématiquement.

La définition d'une fonction est composée :

- du mot clé **def**
- le nom de la fonction
- les noms des paramètres de la fonction
- le corps de la fonction

```
def nomDeLaFonction(liste de paramètres):
```

```
    ...
```

```
    bloc d'instructions
```

```
    ...
```

```
[resultat=.....] # [...] c'est optionnel
```

```
[return resultat]
```

Chapitre 2:

Rappel des bases du langage python

Utilisation des fonctions dans une autre fonction:

Les fonctions peuvent aussi s'utiliser dans des scripts. Veuillez essayer vous-même le petit programme ci-dessous, lequel calcule le volume d'une sphère à l'aide de la formule : $V = \frac{4}{3} * \pi * r^3$

```
def cube(n):  
    return n**3  
  
def volumeSphere(r):  
    return 4 * 3.1416 * cube(r) / 3  
  
r = float(input('Entrez la valeur du rayon : '))  
print ('Le volume de cette sphère vaut', volumeSphere(r))
```

Ce programme comporte trois parties : les deux fonctions **cube()** et **volumeSphere()**, et ensuite le corps principal du programme.

Dans le corps principal du programme, il y a un appel de la fonction **volumeSphere()**.

A l'intérieur de la fonction **volumeSphere()**, il y a un appel de la fonction **cube()**.

Chapitre 2:

Rappel des bases du langage python

Fonction anonyme (lambda function)

Le mot-clé **lambda** en Python permet la création de **fonctions anonymes** (non définie par **def**)

Exemple:

```
>>> f = lambda x : x * x
>>> f ( 3 )
9
```

On peut également préciser plusieurs arguments:

```
>>> g = lambda x , y : x * y
>>> g(5, 7)
35
```

On peut également préciser des valeurs par défaut :

```
>>> g = lambda x , y=2 : x * y
>>> g ( 6 )
12
```

Chapitre 2:

Rappel des bases du langage python

1. Fonction lambda simple : calcul du carré

```
carre = lambda x: x * x
```

```
print("Carré de 5 :", carre(5))      # → 25
```

2. Utilisation avec sorted : trier par le 2e élément

```
liste = [(1, 4), (3, 1), (2, 7)]
```

```
triee = sorted(liste, key=lambda t: t[1])
```

```
print("Triage par 2e élément :", triee)  # → [(3, 1), (1, 4), (2, 7)]
```

3. Utilisation avec map : appliquer une opération à chaque élément

```
nombres = [1, 2, 3, 4]
```

```
carres = list(map(lambda x: x * x, nombres))
```

```
print("Carrés :", carres)            # → [1, 4, 9, 16]
```

4. Utilisation avec filter : filtrer les nombres pairs

```
pairs = list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6]))
```

```
print("Nombres pairs :", pairs)      # → [2, 4, 6]
```

Chapitre 2:

Rappel des bases du langage python

En Python, **toute fonction anonyme peut être réécrite avec def**, car `lambda` n'est qu'une manière raccourcie d'écrire une fonction.

`lambda` est idéal pour **une petite opération ponctuelle, qui ne sera pas réutilisée plusieurs fois dans le code.**

```
liste=[10,3,7,4,11,6,9]
#version 1
sorted(liste, key=lambda t: t[1])
print(liste)
```

```
#version 2
def cle_tri(t):
    return t[1]

sorted(liste, key=cle_tri)
print(liste)
```

Chapitre 2:

Rappel des bases du langage python

Fonctions comme valeurs de première classe

En Python, une fonction est une valeur comme une autre, c'est-à-dire qu'elle peut être passée en argument. On dit que les fonctions sont des valeurs de première classe.

Une application directe est la définition d'un opérateur mathématique par une fonction.

Par exemple, la somme : $\sum_{i=1}^n f(i)$, pour une fonction **f** quelconque, peut être ainsi définie :

```
def    somme_fonction( f , n ) :  
    s = 0  
    for i in range(n+1) :  
        s = s + f ( i )  
    return s
```

Chapitre 2:

Rappel des bases du langage python

Exemple d'application:

Pour calculer la somme des carrés des entiers de 1 à 10, on commence par définir une ***fonction carre***.

```
def    carre ( x ) :  
    return x*x
```

Puis, on la passe en argument à la fonction `somme_fonction` :

```
>>> somme_fonction ( carre , 10 )  
385
```

Chapitre 2:

Rappel des bases du langage python

Fonctions récursives

Une fonction est dite récursive si elle s'appelle elle-même : on parle alors d'appel récursif de la fonction.

Exemple : Fonction factorielle

La fonction factorielle qui calcule le produit des n premiers entiers positifs:

$(n! = \prod_{k=1}^n k)$ est simplement définie par la relation de récurrence suivante:

$$\begin{cases} 0! = 1 \\ n! = n.(n-1)! \end{cases} \quad \forall n > 0$$

Version itérative :	Version récursive :
<pre>def factorielle(n) : f = 1 for k in range(2,n+1) : f = f * k return f</pre>	<pre>def factorielle(n) : if n==0 : return 1 else: return n*factorielle(n-1)</pre>

Chapitre 2:

Rappel des bases du langage python

Variable locale et variable globale

1. Variable locale

Les variables déclarées dans une fonction sont uniquement visibles à l'intérieur de cette fonction. On dit que ce sont des **variables locales**.

2. Variable globale

Les variables déclarées à l'extérieur de toutes les fonctions sont disponibles à toutes les fonctions du programme par l'utilisation du mot clé **global**. Ce sont alors des **variables globales**.

Conseils :

- Les variables globales sont à utiliser avec précaution.
- Il faut faire attention à ne pas cacher involontairement des variables globales par des variables locales portant le même nom.
- Il est conseillé *d'écrire des programmes aussi localement que possible*.

Chapitre 2:

Rappel des bases du langage python

```
1 def f(t):
2     x=t/2
3     return x**2
4
5 def g(t):
6     global x
7     x=t/2
8     return x**2
9
10 #####
11 x=10
12 print("la valeur de x est:",x)
13 y=f(x)
14 print("la valeur de x est:",x," et la valeur de y est:",y)
15 y=g(x)
16 print("la valeur de x est:",x," et la valeur de y est:",y)
17
```

Chapitre 2:

Rappel des bases du langage python

```
1 def f(t):
2     x=t/2
3     return x**2
4
5 def g(t):
6     global x
7     x=t/2
8     return x**2
9
10 #####
11 x=10
12 print("la valeur de x est:",x)
13 y=f(x)
14 print("la valeur de x est:",x," et la valeur de y est:",y)
15 y=g(x)
16 print("la valeur de x est:",x," et la valeur de y est:",y)
17
```

```
la valeur de x est: 10
la valeur de x est: 10 et la valeur de y est: 25.0
la valeur de x est: 5.0 et la valeur de y est: 25.0
```

Exercice d'application:

- 1) En utilisant une boucle while, écrire la fonction **def f(a)**, qui à un réel strictement positif a associe : $\min \{n \in \mathbb{N}, 2^n > a\}$

#assert a>0, "Attention a doit être positif"

- 1) Que pensez-vous du code suivant et quel est son résultat :

```
for i in range(1,4.5,0.5):  
    print(i*10)
```

- 3) Que pensez-vous du code suivant et quel est son résultat :

```
for i in range(10,40,-5):  
    print(i*10)
```

- 4) Que pensez-vous du code suivant et quel est son résultat :

```
for i in range(10,10,1):  
    print(i*10000)
```